

Globant ▶

SENA Soft 2025 Synthetic Edition

Construyendo el futuro del trabajo:
humanos y máquinas inteligentes



Conoce las *fases* de esta aventura dividida en *capítulos*



1 al 5
Agosto

El Despertar del Equipo Agéntico

6 al 29
Agosto

Aprendices

1
Agosto

El Concilio de los Sabios

La Batalla del MVP

Ruta Habilitadora ✨

¿MCP es el nuevo REST?

Agenda

01 Introducción

02 El problema
con los LLMs

03 ¿Que es un
MCP?

04 La
Arquitectura
de los MCP

05 Usando un
MCP

06 Creando un
MCP

07 Pasos
Siguietes:
Seguridad

08 Pasos
Siguietes:
A2A

¡Están a punto de anunciarse los seleccionados!

Alan
Jurado Sintético



1. Los prompts y configuración de la selección serán subidos a GitHub: `rubric.md`, `calibrate.json` y `admission_prompts_alan.csv`.
2. Estamos terminando de procesar algunos puntajes pues la IA da un primer borrador pero recuerden que eso se valida posteriormente (Human-In-The-Loop).
3. El procesamiento está tardando más de lo planeado pues se está formateando todo para dejarlo de consulta en la caja blanca (repo de GitHub).
4. Hoy esperamos anunciar la mayor cantidad de seleccionados posible. Recuerden que en total serán **45 aprendices**.

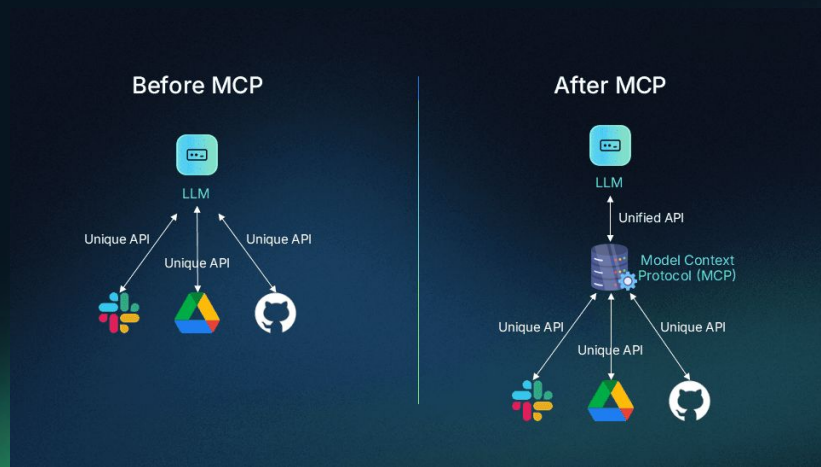
El Problema con los LLMs

Los modelos de lenguaje grandes (LLM) por sí solos, aunque potentes para conversar, tienen limitaciones inherentes que llevaron al desarrollo del Protocolo de Contexto del Modelo (MCP):

Hacia finales de 2024, Anthropic lanzó el MCP para **transformar los chatbots de simples conversadores en agentes que pueden actuar**. Esto indica que los LLM necesitaban una capa adicional para ir más allá de la mera interacción conversacional. Un LLM base tiene una **fuentes de información estática** derivada de sus datos de entrenamiento. Para tareas complejas, se necesitaba algo que pudiera:

- Acceder a **nueva información**.
- **Actuar sobre su entorno** a través de herramientas.
- Operar de forma **semi-autónoma**.
- **Planificar y tomar decisiones**.
- **Coordinarse y colaborar** con otros agentes.

El MCP surge para estandarizar la forma en que los LLM utilizan herramientas y acceden a recursos, permitiendo que los chatbots estándar se conviertan en agentes completamente funcionales



¿Qué es un MCP?

- El **Model Context Protocol (MCP)** es un protocolo que estandariza la forma de transformar los chatbots de simples conversadores en **agentes capaces de actuar**.
- Proporciona una **interfaz común** para construir herramientas, *prompts* y recursos de datos, y conectarlos a los LLM.
- Su objetivo es permitir que los chatbots de IA generativa se conviertan en **agentes completos que pueden actuar, planificar, usar herramientas y colaborar en tareas complejas**.
- Los **clientes MCP** actúan como la interfaz entre su aplicación, el servidor MCP y el LLM, gestionando la comunicación.
- Los **servidores MCP** pueden proporcionar varias capacidades, incluyendo:
 - **Recursos**: datos como archivos de texto o registros.
 - **Prompts**: definiciones de *prompts* dinámicos para los LLM.
 - **Herramientas**: código ejecutable que un agente o modelo puede ejecutar.
 - **Sampling**: una forma de solicitar finalizaciones de chat del modelo de la aplicación.
 - **Imágenes**.
 - **Contexto**: capacidades MCP incorporadas para herramientas.
- Las **tres primitivas principales del MCP** son herramientas, *prompts* y recursos.



¿Qué no es un MCP?

- El MCP no es un LLM en sí mismo, sino un **protocolo que potencia los LLM** para operar como agentes.
- No es una aplicación de *chatbot* por sí misma, sino el **mecanismo subyacente que permite a las aplicaciones *host* integrar capacidades de agente**.
- No se limita a un solo modelo, ofreciendo **agnosticismo de modelo** para el desarrollador de la aplicación.

● MCP

El Protocolo de Contexto de Modelo es un paradigma conversacional y con estado, diseñado para interacciones dinámicas y en tiempo real con agentes de IA.

Contexto: Stateful

Control: Dinámico (LLM)

Ideal para: Agentes de IA

● REST

Transferencia de Estado Representacional es el estándar web sin estado, basado en recursos y operaciones estándar (CRUD).

Contexto: Stateless

Control: Fijo (Servidor)

Ideal para: APIs web CRUD

● GraphQL

Un Lenguaje de Consulta para APIs que permite a los clientes solicitar exactamente los datos que necesitan, minimizando la sobrecarga.

Contexto: Stateless

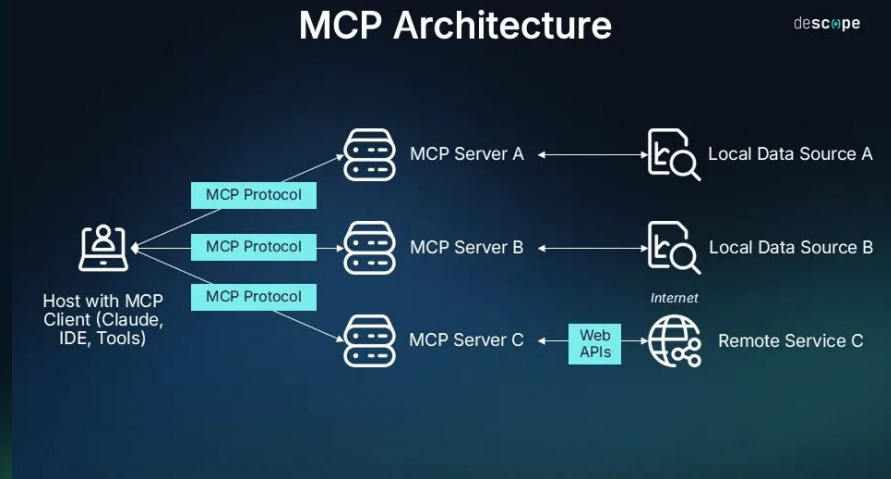
Control: Flexible (Cliente)

Ideal para: Frontends complejos

La Arquitectura de los MCP

La arquitectura de MCP se compone de varios elementos clave que trabajan juntos para habilitar la agenticidad en los LLM:

- **Aplicación *Host***
- **Cliente MCP**
- **Servidor MCP**
- **Transportes**



La Arquitectura de los MCP

- **Aplicación *Host*:**

- Es cualquier aplicación que **aloja uno o más clientes MCP** y se comunica con un LLM.
- Puede ser desde un simple *script* de *chatbot* hasta un IDE completo como Cursor o Windsurf.
- Su rol principal es **interactuar con un LLM y alojar el cliente MCP** para acceder a las capacidades del servidor.



La Arquitectura de los MCP

Cliente MCP:

- Es uno de los componentes más importantes, actuando como la **interfaz de comunicación entre la aplicación *host*, el servidor MCP y el LLM.**
- Un cliente individual solo puede conectarse a **un único servidor**. Para múltiples servidores, se gestionan múltiples instancias de cliente o se utiliza `ClientSessionGroup`.
- Las tareas básicas del cliente incluyen **conectarse al servidor, descubrir sus recursos y ponerlos a disposición del LLM.**
- Puede soportar funciones como **autenticación, filtrado de recursos y agnóstico de modelo.**
- Los clientes también pueden proporcionar **capacidades al servidor**, como el *sampling* (permitiendo que el servidor solicite finalizaciones de chat del LLM del cliente) y *roots* (indicando al servidor a qué áreas del sistema de archivos tiene acceso).



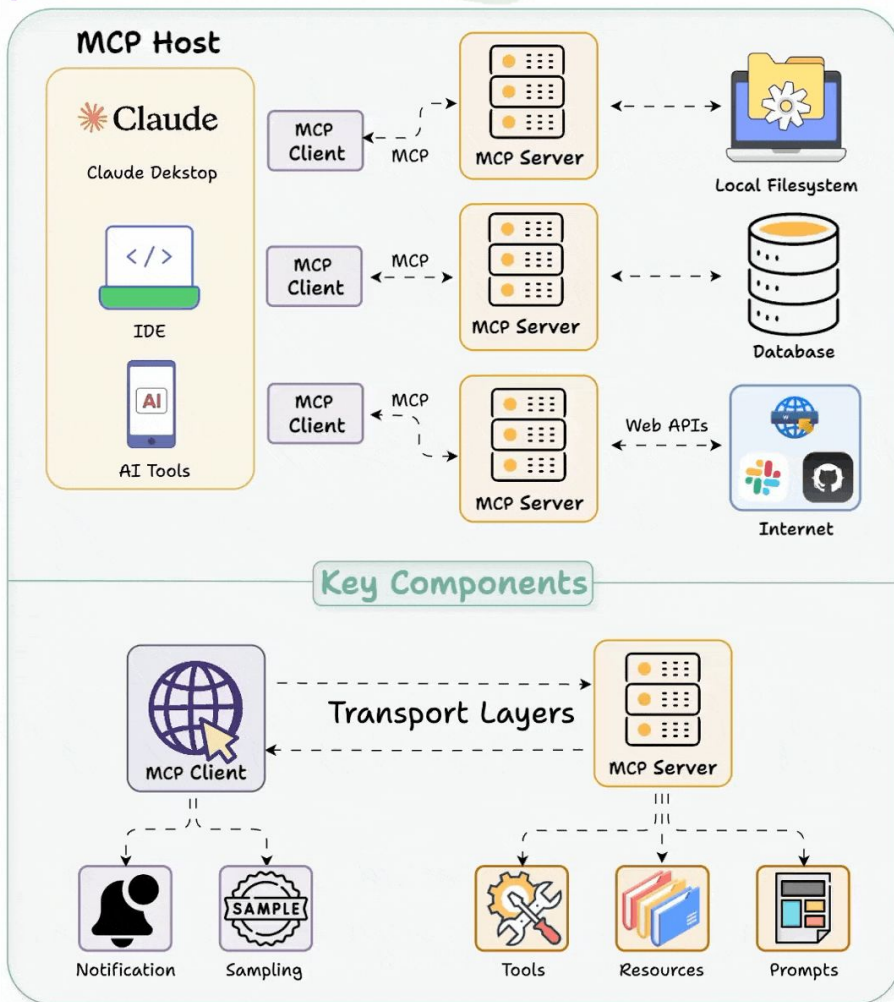
La Arquitectura de los MCP

Servidor MCP:

- Es una pieza principal de la arquitectura, diseñada para ser **sencilla de iniciar, pero potente para flujos de trabajo avanzados**.
- Proporciona las **primitivas MCP**: herramientas, *prompts* y recursos, así como *sampling*, imágenes y contexto.
- Muchos **servidores MCP son creados por usuarios** para aumentar los flujos de trabajo de IA generativa.



Model Context Protocol (MCP), clearly explained



La Arquitectura de los MCP

- **Transportes:**

- Son las implementaciones de la capa de transporte del protocolo que **gestionan cómo se envían los mensajes entre el cliente y el servidor.**

- Los dos principales transportes integrados son:

- **Standard Input/Output (stdio):** Ideal para servidores que se ejecutan junto con la aplicación *host*, utilizando flujos de entrada y salida estándar.

- **Streamable HTTP:** Adecuado para servidores MCP remotos que no se espera que se ejecuten con la aplicación *host* (por ejemplo, plataformas o herramientas alojadas). Permite respuestas HTTP estándar o *streaming* SSE opcional. La seguridad es crucial para este transporte remoto.

- **ClientSessionGroup:** Una clase del SDK de Python que **gestiona múltiples sesiones de cliente concurrentemente**, centralizando el acceso a las primitivas (herramientas, recursos, *prompts*) ofrecidas por varios servidores conectados



MCPs populares en este momento

12 MCP Servers You Can Use in 2025

ByteByteGo

1 File System Server



Gives the LLM direct access to the local file system to read, write, create directories

2 GitHub MCP Server



Connects Claude to GitHub repos and allows file updates, code searching

3 Slack MCP Server



MCP Server for Slack API, enabling Claude to interact with Slack workspaces

4 Google Maps MCP Server



MCP Server for Google Maps API

5 Docker MCP Server



Integrate with Docker to manage containers, images, volumes, and networks

6 Brave MCP Server



Web and local search using Brave's Search API

7 PostgreSQL MCP Server



A MCP Server that enables LLM to inspect database schemas and execute read-only queries

8 Google Drive MCP Server



MCP Server integrates with Google Drive to allow reading and searching over files

9 Redis MCP Server



MCP Server that provides access to Redis databases

10 Notion MCP Server



This project implements an MCP Server for the Notion API

11 Stripe MCP Server



MCP Server to interact with the Stripe API

12 Perplexity MCP Server



An MCP Server that connects to Perplexity's Sonar API for real-time search

Usando un MCP



Creando un MCP



Seguridad

- **Protección de credenciales:** Es crucial **no comprometer archivos** .env en el control de versiones, ya que exponen claves API. Las claves comprometidas deben ser desautorizadas inmediatamente.
- **Seguridad de transporte remoto:** Los transportes remotos, como Streamable HTTP, **deben ser debidamente asegurados** mediante autenticación de conexión y validación de encabezados de origen para prevenir problemas de seguridad.
- **Aprobación humana (Human-in-the-Loop - HITL):** Dada la capacidad de los servidores externos para solicitar acceso al LLM de la aplicación (con posibles costes), **se recomienda encarecidamente implementar un proceso de aprobación humana** antes de permitir que un servidor envíe una solicitud al LLM a través de su cliente. La autenticación es una característica importante a implementar en el cliente para esto



MCP Remotos

- **Conectividad flexible:** El transporte **Streamable HTTP** está diseñado específicamente para **servidores MCP remotos**, permitiendo que las aplicaciones *host* se conecten a servidores que no se ejecutan localmente.
- **Infraestructura alojada:** Las organizaciones están explorando el **alojamiento de servidores MCP en sus propias redes** y exponiéndolos públicamente. Los clientes pueden integrarse con ellos utilizando Streamable HTTP.
- **Gestión de múltiples conexiones:** La clase `ClientSessionGroup` en el SDK de Python facilita la **conexión a múltiples servidores**, lo cual es fundamental para interactuar con una red de servicios MCP remotos



Agente a Agente (Agent to Agent)

- **Colaboración de agentes:** Una de las capacidades más prometedoras que los agentes MCP habilitan es la **capacidad de los agentes para trabajar juntos y comunicarse entre sí** para realizar tareas complejas.
- **Avances en investigación:** Existe un gran interés en investigar cómo lograr que los agentes colaboren de manera efectiva.
- **Información futura:** Aunque el potencial para la colaboración entre agentes es un área de gran interés, las fuentes indican que **hay más contenido por venir** sobre este tema en futuras entregas del libro. Por lo tanto, los detalles específicos sobre arquitecturas o patrones de interacción agente-agente no están completamente cubiertos en este momento.



